Neural Networks

04 — Convolutional Neural Networks
CosmicAI Summer School 2026

The parameter problem

```
# A modest image: 1000×1000 pixels, RGB colour
1000 * 1000 * 3          # = 3,000,000 inputs

# A hidden layer with 1,000 neurons
1000 * 1000 * 3 * 1000   # = 3,000,000,000 weights
```

3 billion parameters — for the first layer alone.

> ⚠ Warning
>
> This is not primarily a storage problem. It is a **generalisation** problem: a fully-connected layer cannot share what it learns about a galaxy at one position with the same galaxy at another position.

What fully-connected layers miss

A spiral galaxy at pixel (100, 100) and the same galaxy at pixel (800, 800):

- **identical in morphology**
- **completely different inputs** to `nn.Linear`

Every position requires its own weights. No knowledge transfers across the image.

```python
class GalaxyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(
            256 * 256 * 3,   # 196,608 inputs
            512,
        )
        # 196,608 × 512
        #   = 100,663,296 weights
        #   … just for layer 1
```

Translation invariance

A useful detector for galaxy spiral arms should fire whether the arm appears:

- at the top of the image
- at the bottom
- anywhere else

> **Note**
>
> **The key idea:** apply the *same* small filter everywhere.
> If a filter detects an edge, it detects that edge regardless of position. The filter learns the pattern; the image tells it where.

Translation invariance and weight sharing are the two properties that convolutional layers encode by construction.

Convolution by hand — 1D

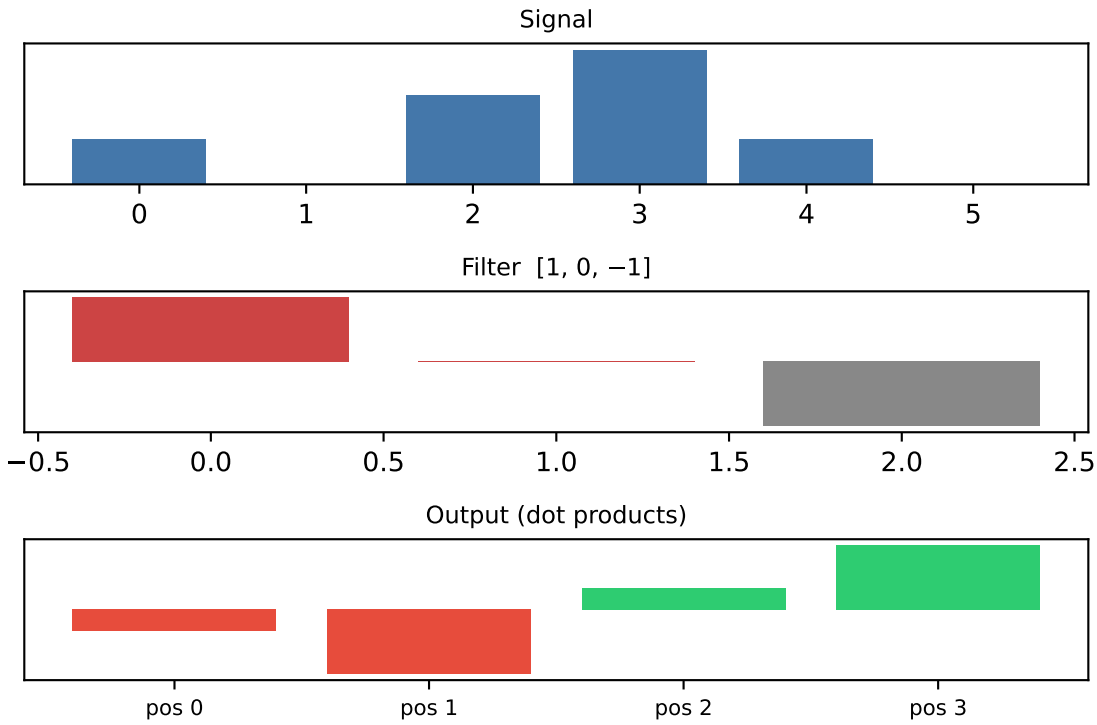
A 1D signal and a 3-element **edge-detecting** filter:

$$\text{filter} = [1,\ 0,\ -1]$$

At each position, compute the dot product of filter with the signal window beneath it.

Output length: $L_{out} = L_{in} - k + 1$

`nn.Conv1d` does this simultaneously at every position — filter weights are learned from data.



2D convolution

A $k \times k$ filter slides over a 2D image.

Output size (no padding, stride 1):

$$H_{out} = H_{in} - k + 1$$
$$W_{out} = W_{in} - k + 1$$

General formula (padding p, stride s):

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

Setting $p = \lfloor k/2 \rfloor$ gives $H_{out} = H_{in}$ ("same" padding).

```python
nn.Conv2d(
    in_channels=3,    # RGB
    out_channels=32,  # 32 filters
    kernel_size=3,    # 3×3
    padding=1,        # same size out
)
# Parameters:
# 3 × 3 × 3 × 32 + 32 = 896
```

One filter learns one feature. 32 filters learn 32 different features — all from the same spatial positions.

Parameter sharing

Fully-connected, first layer:

$256 \times 256 \times 3$ inputs, 512 outputs $\rightarrow$ **100 M weights**

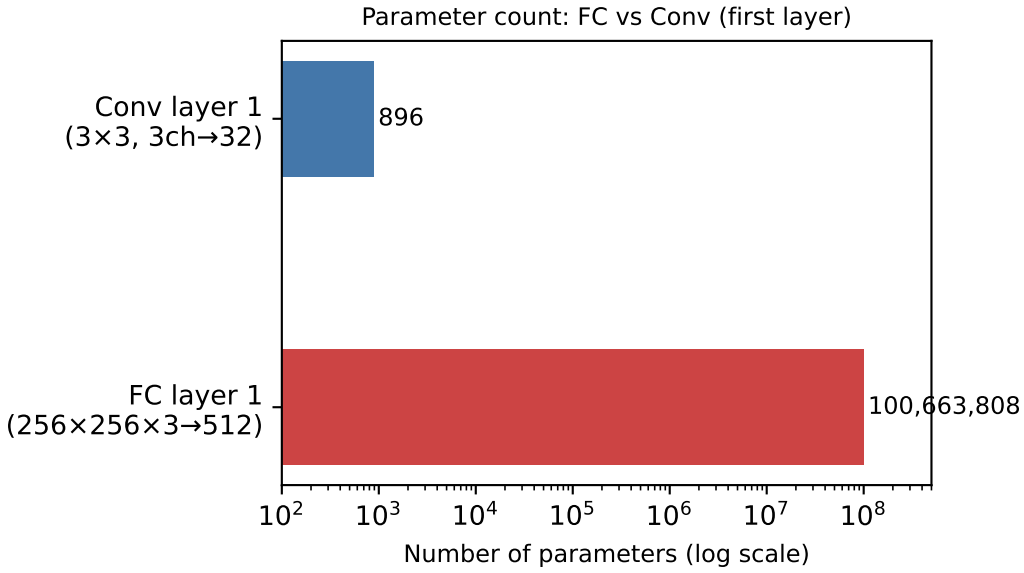
Each neuron looks at the whole image.

Convolutional, first layer:

3×3 filter, 3 ch in, 32 out $\rightarrow$ **896 weights**

The same 896 weights scan every position.

A CNN encodes the prior: **the pattern matters, not its position.**



Pooling

After convolution, reduce spatial resolution deliberately:

```
nn.MaxPool2d(kernel_size=2, stride=2)
# 256×256 → 128×128  (halves each spatial dimension)
```

What pooling does:

- Discards exact position information
- Retains *whether* a feature was detected in that region
- Reduces computation for all subsequent layers

Spatial dimensions after 3× MaxPool2d(2,2) on a 256×256 image:

$$256 \rightarrow 128 \rightarrow 64 \rightarrow 32$$

The classifier head then sees a 32×32 feature map — 64× smaller than the input.

What filters learn — hierarchical features

Layer	What filters detect	Astronomy analogy
Layer 1	Edges, colour gradients	Limb of a galaxy disc, star PSF
Layer 2	Textures, curved strokes	Spiral arm segments, bar
Layer 3	Object parts	Bulge, ring, extended emission
Layer 4	Whole objects	Morphological class

> **Note**
>
> Zeiler & Fergus (2013) visualised this by projecting learned filter weights back into pixel space. The hierarchy **emerges from the data** — it is not hand-designed.

You will reproduce a version of this visualisation in **Notebook 05**.

GalaxyCNN: a concrete architecture

```python
class GalaxyCNN(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3,   32, 3, padding=1), nn.BatchNorm2d(32),  nn.ReLU(),
            nn.MaxPool2d(2),                                  # 256 → 128
            nn.Conv2d(32,  64, 3, padding=1), nn.BatchNorm2d(64),  nn.ReLU(),
            nn.MaxPool2d(2),                                  # 128 → 64
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.MaxPool2d(2),                                  # 64  → 32
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(128 * 32 * 32, 256), nn.ReLU(),    # 131,072 → 256
            nn.Linear(256, n_classes),
        )
```

Total parameters: ~8.5 M — versus ~100 M for an equivalent fully-connected network.

Astronomy applications

Task	Architecture	Example dataset
Galaxy morphology classification	CNN (3–5 blocks)	Galaxy10 DECaLS, GalaxyZoo
RFI detection	CNN on time–freq spectrograms	MeerKAT, LOFAR
Weak gravitational lensing	CNN on survey images	DES, Euclid shear maps
Transient / alert classification	CNN on difference images	ZTF, Rubin LSST
Pulsar candidate classification	CNN on fold profiles	HTRU, PALFA
Source finding	CNN / U-Net segmentation	SKA precursor surveys

> **Note**
>
> All of these tasks share the same structure: local patterns at arbitrary positions that a fully-connected layer cannot generalise across.

Notebook 05 — what you will build

In the next 35 minutes:

1. **Convolution by hand** — step a 1D kernel across a stellar spectrum; watch the output accumulate
2. **2D convolution on a galaxy image** — select an edge, blur, or sharpen kernel; compare input and filtered output
3. **Feature maps** — choose a filter channel in a trained network; see what it detects on a galaxy image
4. **Parameter count arithmetic** — the numbers from this slide, in runnable code
5. **Pooling** — trace spatial dimensions through max and average pooling

Exercise: implement `conv2d_step(patch, kernel)` for a single spatial position; verify against `F.conv2d`.

References

Papers

- Zeiler & Fergus (2013) — *Visualizing and Understanding Convolutional Networks.* ECCV. arxiv.org/abs/1311.2901
- LeCun et al. (1998) — *Gradient-based learning applied to document recognition.* Proc. IEEE.

Blogs

- Karpathy — *CS231n: CNNs for Visual Recognition* cs231n.github.io/convolutional-networks/
- Colah — *Conv Nets: A Modular Perspective* colah.github.io/posts/2014-07-Conv-Nets-Modular/